

개발 환경 세팅 가이드

저장소를 처음 받은 신입 · 주니어 개발자를 위한 상세 단계별 안내서

대상 스택

Spring Boot 4.1 · React 19 + Phaser 4 · MySQL 8.0

예상 소요 시간

30 ~ 50분 (도구 설치 포함)

기준 환경

Windows · SourceTree + Git Bash

이 문서는 무엇을 다루나요? 저장소를 처음 clone/pull 받은 뒤 **로컬에서 세 개 서버(backend-llm-proxy-frontend)를 오류 없이 띄우는 것**까지를, 개발 도구를 처음 만져보는 사람도 그대로 따라갈 수 있도록 아주 잘게 쪼개서 설명합니다. 각 단계마다 “성공했을 때 화면/로그”와 “실패했을 때 화면/로그”를 함께 보여주니, 뭔가 이상하면 다음 단계로 넘어가지 말고 그 자리에서 비교해보세요. 게임 로직·LLM 연동·DB 엔티티 설계 등 실제 기능 구현은 다루지 않습니다. 저장소 루트의 `README.md` 도 함께 참고하세요.

1 사전 준비물 설치

2 저장소 받기 (SourceTree)

3 환경 변수 파일 준비

4 로컬 MySQL 실행 (Docker)

5 Backend 실행

6 LLM Proxy 실행

7 Frontend 실행

8 최종 체크리스트

9 로컬 DB vs 팀 공유 DB

10 에러 메시지 빠른 참조표

11 자주 발생하는 문제 (상세)

12 자주 쓰는 명령어 모음

13 그래도 막히면

1 사전 준비물 설치

아래 다섯 가지를 순서대로 설치합니다. 이미 설치되어 있다면 해당 항목의 확인 명령어만 실행해서 버전을 확인하고 넘어가세요. **설치 중간에 새 터미널을 열어야 인식되는 경우가 많으니**, 하나 설치할 때마다 열려있던 터미널을 모두 닫고 새로 열어서 확인하는 습관을 들이세요.

1-1. Git

공식 사이트(git-scm.com)에서 Windows용 설치 파일을 받아 기본 옵션 그대로 설치합니다. 설치 옵션 화면에서 특별히 건드릴 것은 없고, 계속 **Next**만 눌러도 됩니다.

```
git --version
# 예: git version 2.4x.x.windows.1
```

1-2. SourceTree

공식 사이트(source-treeapp.com)에서 설치 후 실행합니다. 처음 실행하면 Atlassian 계정 로그인 또는 “건너뛰기”를 묻는데, 계정 없이 **건너뛰기**를 선택해도 이 프로젝트를 쓰는 데는 문제없습니다. 이어서 “Git을 찾을 수 없다”는 안내가 나오면 **시스템에 설치된 Git 사용**을 선택하세요 (1-1에서 이미 설치했으므로).

1-3. Java 21 (JDK)

Eclipse Temurin(adoptium.net)에서 **21 (LTS)** 버전, Windows x64, `.msi` 설치 파일을 받습니다. 설치 화면에서 **“Set JAVA_HOME variable”, “Add to PATH”** 옵션이 보이면 반드시 체크하세요 (기본으로 꺼져 있는 경우가 있습니다). 이 옵션을 켜야 터미널에서 `java` 명령이 바로 인식됩니다.

```
java -version
# 예: openjdk version "21.0.x" ...
```

버전이 21이 아니거나 명령어를 찾을 수 없다는 오류가 뜨면, 터미널을 전부 닫고 새로 열어서 다시 확인하세요. 그래도 안 되면 PC를 재시작하세요 (PATH 환경변수가 새로 로드됩니다).

1-4. Node.js 24.15.0

이 프로젝트는 정확히 **24.15.0** 버전을 기준으로 합니다(`frontend/.nvmrc`). 버전 관리가 필요 없다면 nodejs.org에서 24.x 버전을 그냥 설치해도 되지만, 다른 프로젝트와 버전이 겹칠 수 있으니 **nvm-windows**(github.com/coreybutler/nvm-windows) 사용을 권장합니다.

```
nvm install 24.15.0
nvm use 24.15.0
node -v
# 예: v24.15.0
```

1-5. Docker Desktop

공식 사이트(docker.com/products/docker-desktop)에서 설치합니다. 설치 중 WSL2 관련 안내가 나오면 기본값 그대로 진행하면 됩니다. 설치가 끝나도 자동으로 켜지지 않는 경우가 많으니, 시작 메뉴에서 **Docker Desktop**을 직접 실행하세요.

```
docker --version
# 예: Docker version 2x.x.x
```

Docker Desktop은 “설치”와 “실행”이 별개입니다

Windows 로그인 시 자동으로 켜지지 않는 경우가 많습니다.

작업을 시작할 때마다

Docker Desktop 앱이 켜져 있는지, 우측 하단 트레이 아이콘이 “Engine running” 상태인지(약 10~30초 소요) 먼저 확인하는 습관을 들이세요. 꺼진 채로 4단계를 진행하면 아래처럼 연결 오류가 납니다.

```
error during connect: this error may indicate that the docker daemon is not running...
```

도구	요구 버전	확인 명령어
Git	제한 없음	<code>git --version</code>
SourceTree	최신	실행해서 메인 화면 확인
Java (JDK)	21 (LTS)	<code>java -version</code>
Node.js	24.15.0	<code>node -v</code>
Docker Desktop	최신, 실행 중	<code>docker --version</code> + 트레이 아이콘 확인

2 저장소 받기 (SourceTree)

2-1. Clone

SourceTree 실행 → 좌측 상단 **New**(또는 **Clone/New**) 클릭 → **Clone from URL** 선택 → 저장소 URL 입력 → **Destination Path**(로컬에 저장할 폴더, 예: `C:\dev\game-project`) 지정 → **Clone** 클릭.

확인 포인트 — Clone이 잘 됐다면

탐색기로 Destination Path 폴더를 열었을 때 아래처럼 보여야 합니다. 이 목록이 하나라도 빠져 있으면 Clone이 중간에 실패한 것이니 폴더를 지우고 2-1부터 다시 하세요.

```
frontend/  backend/  llm-proxy/  docs/
docker-compose.yml  .env.example  README.md  .gitignore
```

2-2. 이미 clone된 저장소를 최신으로 받을 때 (재방문 시)

SourceTree에서 해당 저장소 탭을 열고, 상단 톨바의 **Pull** 버튼 클릭. `.env`, `application-local.yml` 같은 로컬 전용 파일은 `.gitignore` 대상이라 Pull을 아무리 반복해도 덮어써지거나 삭제되지 않습니다.

3단계부터는 터미널 명령이 필요합니다

SourceTree는 clone/pull/commit 같은 git 작업용 GUI이고, 아래 3~7단계(파일 복사, docker, gradlew, npm)는 명령줄에서 실행해야 합니다. SourceTree 상단 톨바의

Terminal

아이콘을 누르면 현재 저장소 폴더가 열린 터미널이 바로 뜹니다 — 매번 `cd` 로 폴더를 찾아갈 필요 없이 이걸 쓰면 편합니다. 기본 연결된 터미널이 Git Bash면 이 문서의 명령어를 그대로 쓰면 되고, cmd/PowerShell이면 `gradlew.bat` 을 쓰세요 (11단계 참고).

3 환경 변수 파일 준비

비밀번호 같은 민감한 값은 git에 올라가면 안 되기 때문에, 저장소에는 `.example` 접미사가 붙은 “템플릿” 파일만 들어 있습니다. 이걸 복사해서 실제 파일을 각자 로컬에 만들어야 합니다.

템플릿 (git에 있음)	내가 만들 파일 (git에 없음, 로컬 전용)	위치
<code>.env.example</code>	<code>.env</code>	저장소 최상위 루트
<code>backend/application-local.yml.example</code>	<code>backend/application-local.yml</code>	<code>backend/build.gradle</code> 과 같은 위치 (src 폴더 안이 아님)
<code>llm-proxy/application-local.yml.example</code>	<code>llm-proxy/application-local.yml</code>	<code>llm-proxy/build.gradle</code> 과 같은 위치 (src 폴더 안이 아님)

3-1. 파일 3개 복사 (SourceTree Terminal 또는 Git Bash에서, 저장소 루트에서 실행)

```
cp .env.example .env
cp backend/application-local.yml.example backend/application-local.yml
cp llm-proxy/application-local.yml.example llm-proxy/application-local.yml
```

PowerShell/cmd라면 `cp` 대신 `copy` 를 써도 됩니다.

파일을 열어 편집할 때 “메모장으로 새로 만들기”는 하지 마세요

만드시 위 `cp` 명령으로 먼저 파일을 만든 뒤,

그 파일을 열어 편집

하세요. 메모장에서 “새로 만들기 → 다른 이름으로 저장”을 하면 파일명 끝에 `.txt` 가 몰래 붙어서 (`.env.txt`) 프로그램이 파일을 찾지 못합니다. 편집기는 VS Code, 메모장, Notepad++ 무엇이든 상관없습니다 —

이미 만들어진 파일을 열어(더블클릭 대신 편집기에서 “열기”) 고치고 저장

만 하면 됩니다.

3-2. 루트 `.env` 값 채우기 (docker-compose용 MySQL 계정)

`.env` 파일을 열어 아래처럼 **전체 내용**을 채웁니다. `=` 앞뒤로 공백을 넣지 마세요.

```
MYSQL_DATABASE=game_project
MYSQL_USER=game_user
MYSQL_PASSWORD=local1234
MYSQL_ROOT_PASSWORD=localroot1234
MYSQL_PORT=3306
```

비밀번호는 로컬 전용이라 복잡할 필요 없습니다. 위 예시 값을 그대로 써도 됩니다. 다만 **MYSQL_PASSWORD** 값은 아래 3-3의 **DB_PASSWORD**와 반드시 똑같아야 합니다 — 이 문서 전체에서 가장 많이 실수하는 지점입니다.

3-3. `backend/application-local.yml` 값 채우기

파일을 열어 **전체 내용**을 아래처럼 만드세요 (기존 안내 주석은 지워도 되고 남겨둬도 무방합니다 — `#`으로 시작하는 줄은 무시됩니다). 로컬 Docker MySQL만 쓸 거라면 `DB_HOST` / `DB_PORT` / `DB_SSL_MODE` 는 아예 안 써도 됩니다(기본값 `localhost:3306` 이 자동 적용).

```
DB_USERNAME: game_user
DB_PASSWORD: local1234
```

3-4. `llm-proxy/application-local.yml` 값 채우기

```
DB_USERNAME: game_user
DB_PASSWORD: local1234

LLM_API_KEY: change_me
```

`LLM_API_KEY` 는 아직 실제 LLM 연동 전이라 `change_me` 같은 임의의 문자열이어도 서버 실행에는 문제없습니다.

YAML 파일(.yaml) 작성 시 꼭 지켜야 하는 규칙

application-local.yaml 은 .env 와 문법이 다릅니다. 아래를 어기면 서버가 아예 시작조차 못 하고 알 수 없는 에러를 내뿜습니다.

- 콜론(:) 뒤에 반드시 띄어쓰기 한 칸.

DB_USERNAME:game_user (X) → DB_USERNAME: game_user (O)

- 탭(Tab) 키로 들여쓰기 금지.

들여쓰기가 필요한 경우 스페이스만 사용 (이 파일들은 들여쓰기 없이 한 줄씩 쓰므로 보통 해당 없음)

- 값에 따옴표를 씌우지 않습니다.

DB_PASSWORD: "local1234" (X, 따옴표까지 비밀번호에 포함됨) → DB_PASSWORD: local1234 (O)

- 줄 끝에

보이지 않는 공백

이 남아있지 않은지 주의 (복사/붙여넣기 후 자동완성으로 공백이 붙는 경우가 있음)

✗ 틀린 예

```
DB_USERNAME:game_user
DB_PASSWORD: "local1234"
```

✓ 맞는 예

```
DB_USERNAME: game_user
DB_PASSWORD: local1234
```

4 로컬 MySQL 실행 (Docker)

4-1. 컨테이너 기동 (저장소 루트에서)

```
docker compose up -d
```

처음 실행하면 MySQL 이미지를 내려받느라 1~2분 정도 걸릴 수 있습니다.

확인 포인트 — 다음 단계로 넘어가기 전에

아래 명령으로 상태를 확인하세요. STATUS 칸에 Up 이 보여야 정상입니다.

```
docker ps
```

CONTAINER ID	IMAGE	...	STATUS	PORTS
073a3b2ce273	mysql:8.0	...	Up 5 seconds	0.0.0.0:3306->3306/tcp

이게 안 보이면 backend를 실행해도 어차피 DB 연결에 실패하니, 아래 “포트 충돌” 안내부터 확인하세요.

“포트 3306이 이미 사용 중”이라는 오류가 뜬다면

PC에 MySQL이 이미 따로 설치되어 있는 경우(XAMPP, 개인 MySQL 서버 등) 흔히 발생합니다.

```
Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:3306
-> ...: bind: Only one usage of each socket address...
```

기존 MySQL을 지울 필요 없이, 이 프로젝트의 Docker 쪽 포트만 바꾸면 됩니다.

1. `.env` 의 `MYSQL_PORT=3306` 을 `3307` 로 변경
2. `backend/application-local.yml`, `llm-proxy/application-local.yml` 맨 아래에 각각 `DB_PORT: 3307` 한 줄씩 추가 (3-3, 3-4에서 만든 내용 아래에)
3. `docker compose up -d` 다시 실행

“컨테이너 이름이 이미 사용 중”이라는 오류가 뜬다면

```
Error response from daemon: Conflict. The container name "/game-project-mysql"
is already in use by container "..."
```

같은 PC에 이 프로젝트를 두 곳(다른 폴더 등)에 받아둔 경우 발생합니다. 안 쓰는 쪽 폴더에서 `docker compose down` 을 실행해서 내리세요.

5 Backend 실행

5-1. 새 터미널 창에서

```
cd backend
./gradlew bootRun
```

Windows PowerShell / cmd 창(Git Bash가 아닌 경우)이라면 `.\gradlew.bat bootRun` 을 사용하세요.

처음 실행하면 Gradle이 필요한 라이브러리를 내려받기 때문에 1~3분 정도 걸릴 수 있습니다.

성공 로그 — 이런 줄이 보이면 정상입니다

```
HikariPool-1 - Start completed.
Tomcat started on port 8080 (http) with context path '/'
Started BackendApplication in 2.5 seconds (process running for 2.9)
```

이 터미널은

끄지 말고 그대로 쳐두세요

(서버가 이 창에서 계속 실행 중입니다). 이 로그가 뜬 뒤 브라우저에서 `http://localhost:8080/health` 를 열어 `OK` 가 보이는지 확인하고 나서 6단계로 넘어가세요.

실패 로그 — “Access denied”가 보인다면

```
HHH000247: ErrorCode: 1045, SQLState: 28000
Access denied for user 'game_user'@'...' (using password: YES)
```

`.env`의 `MYSQL_PASSWORD`와 `backend/application-local.yml`의 `DB_PASSWORD`가 서로 다른 경우입니다 (3-2, 3-3 참고). 값을 똑같이 맞춘 뒤 터미널에서 `Ctrl + C`로 종료하고 `./gradlew bootRun`을 다시 실행하세요.

6 LLM Proxy 실행

6-1. 또 다른 새 터미널 창에서

```
cd llm-proxy
./gradlew bootRun
```

성공 로그

```
HikariPool-1 - Start completed.
Tomcat started on port 8081 (http) with context path '/'
Started LlmProxyApplication in 2.7 seconds (process running for 3.0)
```

브라우저에서 `http://localhost:8081/health` → `OK` 확인 후 7단계로 넘어가세요. `backend`와 동일하게 `Access denied`가 보이면 `llm-proxy/application-local.yml`의 `DB_PASSWORD`를 확인하세요.

7 Frontend 실행

7-1. 또 다른 새 터미널 창에서

```
cd frontend
npm install
npm run dev
```

성공 로그

```
VITE v8.2.0 ready in 332 ms
→ Local: http://localhost:5173/
```

브라우저에서 `http://localhost:5173`을 열어 “Hello Phaser” 텍스트가 있는 캔버스 화면이 보이면 성공입니다.

지금까지 총 3개의 터미널 창이 켜져 있어야 정상입니다

backend(8080) · llm-proxy(8081) · frontend(5173) 각각 한 창씩. 하나의 터미널에서 다음 명령을 입력하기 전에는 반드시 새 터미널 탭/창을 열어서 실행하세요. 같은 창에 이어서 입력하면 이미 실행 중인 서버가 멈춥니다.

8 최종 체크리스트

아래 항목을 모두 통과하면 환경 세팅은 끝난 것입니다.

- ☐ `docker ps` 에 `game-project-mysql` 이 `Up` 상태로 표시됨
- ☐ 브라우저에서 `http://localhost:8080/health` 접속 → `OK` 텍스트가 보임
- ☐ 브라우저에서 `http://localhost:8081/health` 접속 → `OK` 텍스트가 보임
- ☐ 브라우저에서 `http://localhost:5173` 접속 → "Hello Phaser" 텍스트가 있는 캔버스 화면이 보임
- ☐ 터미널 3개(backend/llm-proxy/frontend)가 에러 없이 계속 실행 중

9 로컬 DB vs 팀 공유 DB

이 프로젝트는 두 가지 DB 접속 방식을 모두 지원합니다. 처음 세팅할 때는 **로컬 Docker MySQL**(위에서 이미 완료)을 쓰면 되고, 팀원들과 같은 데이터를 봐야 할 때만 공유 DB로 바꾸면 됩니다.

	로컬 Docker MySQL	팀 공유 DB (Aiven 등)
언제 사용	혼자 기능 개발/테스트할 때 (기본값)	팀원들과 같은 데이터를 보며 통합 테스트할 때
설정 방법	<code>application-local.yml</code> 에 <code>DB_USERNAME</code> / <code>DB_PASSWORD</code> 만 채우면 끝	같은 파일에 <code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_SSL_MODE: REQUIRED</code> 추가
접속 정보	본인이 3-2에서 정한 값	팀 채널/시크릿 매니저로 전달받은 값 (이 문서 채팅에 평문으로 남기지 않기)

공유 DB 설정 예시 (backend/llm-proxy 공통, 파일 맨 아래에 추가)

```
DB_HOST: 팀에서 전달받은 호스트 주소
DB_PORT: 팀에서 전달받은 포트
DB_SSL_MODE: REQUIRED
```

10 에러 메시지 빠른 참조표

화면에 뜬 에러 메시지에서 눈에 띄는 단어를 아래 표에서 찾아보세요. 자세한 설명은 11단계에 있습니다.

에러 메시지에 이 단어가 보이면	원인	바로가기
Access denied for user	.env와 application-local.yml의 비밀번호 불일치	11-1
ports are not available / bind: ... 3306	PC에 이미 MySQL이 떠 있어 포트 충돌	4단계, 11-2
Address already in use / 8080, 8081, 5173	이전에 켜둔 서버가 아직 실행 중	11-2
container name ... already in use	같은 PC에 프로젝트가 두 곳에 clone됨	4단계, 11-4
cannot connect to the Docker daemon	Docker Desktop이 꺼져 있음	1-5, 11-4
command not found: ./gradlew	Git Bash가 아닌 cmd/PowerShell 사용 중	11-3
Unable to determine Dialect / JDBC	DB(MySQL)에 연결 자체가 안 됨 — 위 항목들 먼저 확인	4, 5, 11-1

11 자주 발생하는 문제 (상세)

11-1. “Access denied for user” 에러가 난다

대부분 `.env`의 `MYSQL_PASSWORD`와 `application-local.yml`의 `DB_PASSWORD`가 서로 다르기 때문입니다. 두 값이 **글자 하나까지 정확히** 같은지 다시 확인하세요. 값을 고쳤다면:

1. 실행 중인 backend/llm-proxy 터미널에서 `Ctrl + C`로 종료
2. `docker compose down`으로 컨테이너를 내렸다가 `docker compose up -d`로 다시 켜기 (비밀번호가 이미 컨테이너 안에 저장된 경우를 대비)
3. `./gradlew bootRun` 재실행

공유 DB에서 새로 계정을 발급받았는데 계속 실패한다면, 계정을 만든 사람에게 비밀번호를 `ALTER USER`로 한 번 더 재설정해 달라고 요청해보세요 (관리형 DB에서 가끔 최초 생성 시 반영이 안 되는 경우가 있습니다).

11-2. 포트가 이미 사용 중이라는 오류 (3306, 8080, 8081, 5173)

다른 프로그램이 그 포트를 쓰고 있다는 뜻입니다. MySQL(3306)은 4단계의 방법대로 포트를 바꾸면 되고, 나머지 포트(8080/8081/5173)는 이미 이전에 켜둔 서버가 남아있는 경우가 대부분입니다. 해당 번호를 쓰던 터미널 창을 찾아 `Ctrl + C`로 종료한 뒤 다시 실행하세요. 어느 창인지 못 찾겠으면 열려있는 터미널 창을 전부 닫고 5~7단계를 처음부터 다시 실행해도 됩니다.

11-3. `./gradlew` 실행이 안 된다 (명령어 없음 / 권한 오류)

Git Bash가 아닌 일반 PowerShell/cmd 창을 쓰고 있다면 `./gradlew` 대신 `gradlew.bat`을 사용하세요.

```
gradlew.bat bootRun
```

SourceTree의 Terminal 버튼으로 연 창이 Git Bash가 아니라면, 위 방법을 쓰거나 시작 메뉴에서 **Git Bash**를 직접 실행해서 `cd`로 폴더까지 이동해 사용하세요.

11-4. Docker 관련 명령어가 전부 연결 오류를 낸다

Docker Desktop 앱이 꺼져 있을 가능성이 가장 높습니다. 1-5의 안내대로 Docker Desktop을 실행하고 트레이 아이콘이 완전히 켜질 때까지(10~30초) 기다린 뒤 다시 시도하세요.

11-5. 브라우저에서 `http://localhost:8080/health` 가 아예 안 열린다 (연결 거부)

backend 터미널 창을 확인하세요. `Started BackendApplication` 로그가 아직 안 났다면 기다리는 중이고, 에러가 떠 있다면 그 에러 메시지를 10단계 표에서 찾아보세요. 터미널 창을 실수로 닫았다면 5단계부터 다시 실행하세요.

11-6. YAML 파일을 고쳤는데도 값이 반영이 안 되는 것 같다

서버를 실행한 채로 파일만 고치면 반영되지 않습니다. **파일 저장 → 해당 터미널에서 `Ctrl` + `C` → 명령어 재실행** 순서를 지켜세요. 저장이 잘 됐는지 헷갈리면 편집기에서 파일을 닫았다가 다시 열어서 값이 그대로인지 확인하세요.

12 자주 쓰는 명령어 모음

목적	명령어
MySQL 컨테이너 켜기	<code>docker compose up -d</code>
MySQL 컨테이너 끄기	<code>docker compose down</code>
MySQL 컨테이너 상태 확인	<code>docker ps</code>
backend 실행	<code>cd backend && ./gradlew bootRun</code>
llm-proxy 실행	<code>cd llm-proxy && ./gradlew bootRun</code>
frontend 실행	<code>cd frontend && npm run dev</code>
frontend 빌드/린트 확인	<code>npm run build</code> / <code>npm run lint</code>
서버 종료	해당 터미널에서 <code>Ctrl</code> + <code>C</code>
Windows에서 <code>./gradlew</code> 대체	<code>gradlew.bat bootRun</code>

13 그래도 막히면

- 저장소 루트의 `README.md` 를 다시 한번 확인하세요.
- 에러 메시지를 그대로 복사해서 팀 채널에 공유하면 훨씬 빨리 도움을 받을 수 있습니다 (비밀번호가 포함된 로그는 가려주세요).
- 어느 단계에서 막혔는지(몇 번 단계, 어떤 명령어)를 함께 알려주면 더 빨리 해결됩니다.
- 이 문서는 “환경 세팅”까지만 다룹니다. 게임 로직/DB 엔티티/LLM 연동은 별도로 안내될 예정입니다.